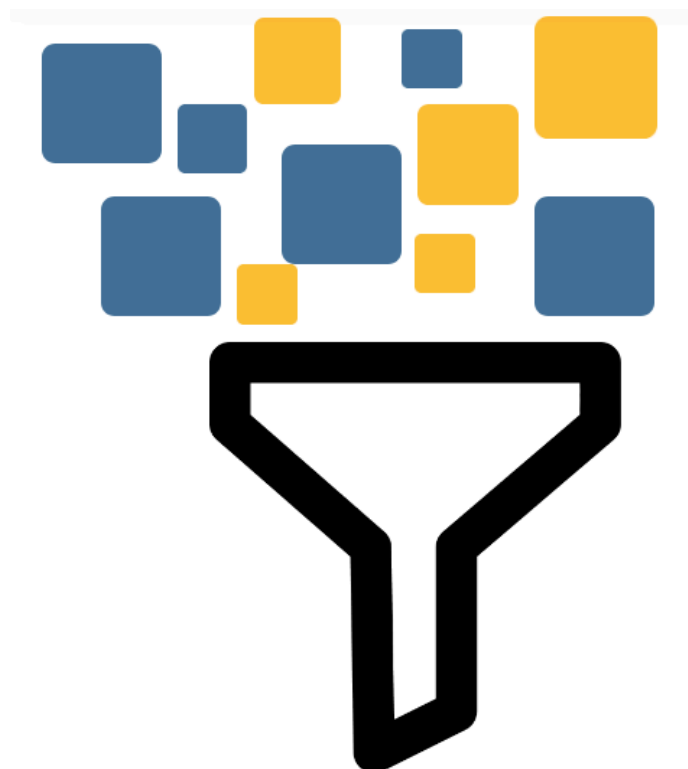


Limpieza y preparación de los datos

Introducción.....	2
Técnicas de limpieza y preparación de datos.....	2
Gestión de los datos que faltan.....	2
Filtrado de datos ausentes.....	2
Rellenado de datos ausentes.....	4
Transformación de datos.....	5
Eliminación de duplicados.....	5
Transformación de datos mediante una función o asignación.....	7
Reemplazar valores.....	8
Discretización.....	8
Detección y filtrado de valores atípicos.....	10
Permutación y muestreo aleatorio.....	11
Agrupación de datos.....	12
Datos tipo datetime.....	13
Apartado de ampliación.....	15
Renombrar índices de eje.....	15
Calcular variables dummy o indicadoras.....	16
Manipulación de cadenas de texto.....	18
Datos categóricos.....	22



Introducción

En el análisis de datos, se emplea una considerable cantidad de tiempo en la preparación de los mismos: carga, limpieza, transformación y reordenación. Contar con información limpia y bien estructurada es esencial para obtener resultados precisos y confiables. Esta fase implica detectar y corregir errores, manejar valores nulos, eliminar duplicados, ajustar formatos y estandarizar la información para garantizar su coherencia.

La limpieza de datos es fundamental para conseguir:

- **Precisión en el Análisis:** Los datos sucios, como los datos incompletos, duplicados o erróneos, pueden distorsionar los resultados del análisis. La limpieza de datos asegura que los resultados sean precisos y fiables.
- **Eficiencia:** Datos bien preparados permiten un análisis más rápido y eficiente. Los analistas pueden centrarse en el análisis propiamente dicho en lugar de lidiar con problemas de calidad de datos.
- **Toma de decisiones informadas:** Decisiones basadas en datos precisos y relevantes mejora la precisión de las decisiones.
- **Cumplimiento y normativas:** La limpieza de datos ayuda a cumplir con normativas y estándares de calidad de datos, evitando posibles sanciones y mejorando la reputación de la empresa.

Técnicas de limpieza y preparación de datos

Gestión de los datos que faltan

En muchas aplicaciones de datos suele haber datos ausentes, faltantes o perdidos. Uno de los objetivos de pandas es que el trabajo con este tipo de datos sea lo más sencillo posible. Algo que se suele hacer es identificar estos valores y eliminarlos o usar técnicas para reemplazarlos. Aunque en apuntes anteriores ya vimos métodos para trabajar con valores nulos, vamos a repasarlos.

Filtrado de datos ausentes

Función	Descripción
dropna	Elimina filas o columnas que contienen valores nulos
isna	Devuelve booleanos indicando si el valor es nulo/ausente
notna	Devuelve booleanos al contrario que isna. Consulta que no sean valores nulos/ausentes

```
import pandas as pd
import numpy as np
```

```
prueba = pd.DataFrame({
    "A": [1, 2, np.nan, 4],
    "B": [5, np.nan, np.nan, 8],
    "C": [9, 10, np.nan, 12]
})
```

	A	B	C
0	1.0	5.0	9.0
1	2.0	NaN	10.0
2	NaN	NaN	NaN
3	4.0	8.0	12.0

```
# Elimina las filas que contengan algún valor nulo
prueba1 = prueba.dropna()
```

	A	B	C
0	1.0	5.0	9.0
3	4.0	8.0	12.0

```
# Elimina las columnas que contengan algún valor nulo
prueba2 = prueba.dropna(axis="columns")
```

```
Empty DataFrame
Columns: []
Index: [0, 1, 2, 3]
```

```
# Solo elimina la fila/columna con todos los valores nulos
prueba3 = prueba.dropna(how="all")
```

	A	B	C
0	1.0	5.0	9.0
1	2.0	NaN	10.0
3	4.0	8.0	12.0

```
# Elimina las filas que contengan más de 2 valores faltantes
prueba4 = prueba.dropna(thresh=2)
```

	A	B	C
0	1.0	5.0	9.0
1	2.0	NaN	10.0
3	4.0	8.0	12.0

Rellenado de datos ausentes

Para la mayoría de los casos se debe emplear el método *fillna*

Argumentos función fillna

Descripción

value

Valor escalar u objeto de tipo diccionario que se utiliza para rellenar valores faltantes

axis

Eje que rellenar. Filas (0 ó "index") o columnas (1 ó "columns")

limit

Determina cuántos valores NaN se pueden rellenar consecutivamente en cada columna o fila antes de detenerse.

```
import pandas as pd
import numpy as np

original = pd.DataFrame(np.random.standard_normal((7,3)))

# Ponemos nulos los valores de las 4 primeras filas de la columna
1
original.iloc[:4,1] = np.nan
# Ponemos nulos los valores de las 2 primeras filas de la columna
2
original.iloc[:2,2] = np.nan
```

	0	1	2
0	0.062692	NaN	NaN
1	-1.379858	NaN	NaN
2	1.934856	NaN	0.930063
3	-0.374948	NaN	0.557342
4	-0.532310	-1.088679	0.649480
5	0.781064	1.857998	0.300983
6	0.031965	-0.518349	-0.973399

```
# Rellenamos los huecos con ceros
rellenado1 = original.fillna(0)
```

	0	1	2
0	0.632250	0.000000	0.000000
1	-1.216332	0.000000	0.000000
2	-1.092250	0.000000	0.878229
3	-0.820047	0.000000	-0.438044
4	1.934321	0.583819	-1.647841
5	0.112707	0.032173	0.855852
6	-1.281958	-1.872129	0.020458

Rellenamos los huecos con diccionario

```
rellenado2 = original.fillna({1: 1.5, 2: 2.5})
```

	0	1	2
0	0.436319	1.500000	2.500000
1	0.293295	1.500000	2.500000
2	0.577954	1.500000	-0.131986
3	-0.780675	1.500000	-1.093722
4	0.473630	-0.127320	1.448386
5	-1.122823	-0.446331	-0.668205
6	0.612272	-1.348449	1.522857

Rellenamos los huecos con interpolación

Ponemos que se rellenen dos huecos máximo

```
rellenado3 = original.bfill(limit=2)
```

	0	1	2
0	-1.516331	NaN	0.669728
1	-0.471286	NaN	0.669728
2	-0.490355	1.733353	0.669728
3	1.117432	1.733353	0.218140
4	-1.586337	1.733353	-0.181955
5	0.596220	-0.212539	-0.713754
6	0.044582	-0.898197	1.243516

Rellenamos los huecos con la media

```
rellenado4 = original.fillna(original.mean())
```

	0	1	2
0	-0.251266	0.703885	-0.107652
1	1.873014	0.703885	-0.107652
2	1.663624	0.703885	-1.679340
3	2.073861	0.703885	-0.273092
4	-0.037209	0.049622	0.239458
5	-1.047683	0.978342	0.114876
6	1.412373	1.083691	1.059837

Transformación de datos

Eliminación de duplicados

La eliminación de duplicados permite evitar redundancias que pueden distorsionar el análisis, asegurando que cada observación sea única y representativa. Existe el método `uplicated()`, que indica si alguna fila es un duplicado devolviendo un `True`, y el `drop_duplicates()`, que elimina registros repetidos de manera eficiente.

```
import pandas as pd

df = pd.DataFrame({
    "A": [1, 2, 3, 1, 2, 3],
    "B": ["X", "Y", "Z", "X", "Y", "Z"],
    "C": [10, 20, 30, 10, 20, 30],
    "D": [100, 200, 300, 100, 200, 300]
})

      A  B   C   D
0  1  X  10  100
1  2  Y  20  200
2  3  Z  30  300
3  1  X  10  100
4  2  Y  20  200
5  4  Z  30  300

# Detectamos duplicados
print(df.duplicated())

      0    False
      1    False
      2    False
      3     True
      4     True
      5    False
      dtype: bool

# Eliminamos filas duplicadas
sin_repes = df.drop_duplicates()

      A  B   C   D
0  1  X  10  100
1  2  Y  20  200
2  3  Z  30  300
5  4  Z  30  300
```

```
# Eliminamos duplicados tomando de referencia una
columna
sin_repes2 = df.drop_duplicates(subset=["B"])
```

	A	B	C	D
0	1	X	10	100
1	2	Y	20	200
2	3	Z	30	300

Transformación de datos mediante una función o asignación

La función *map()* permite aplicar otra función definida previamente a cada elemento de una lista iterable. Es una forma conveniente de realizar transformaciones elemento a elemento y otras operaciones de datos relacionadas con su limpieza.

```
import pandas as pd

aquarium = pd.DataFrame({
    "Salas": [1, 2, 3, 4, 5],
    "Especie": ["Boquerón", "Trucha", "Barbo", "Sardina", "Carpa"],
    "Total peces": [10, 8, 15, 20, 9],
})
```

	Salas	Especie	Total peces
0	1	Boquerón	10
1	2	Trucha	8
2	3	Barbo	15
3	4	Sardina	20
4	5	Carpa	9

```
pez_agua = {
    "Boquerón": "agua salada",
    "Trucha": "agua dulce",
    "Lubina": "agua salada",
    "Dorada": "agua salada",
    "Barbo": "agua dulce",
    "Sardina": "agua salada",
    "Perca": "agua dulce",
    "Carpa": "agua dulce"
}
```

```
# Añadimos columna que indica el tipo de agua del pez
def get_habitat(n):
    return pez_agua[n]
```

```
# Añadimos columna con el tipo de agua
aquarium["Hábitat"] = aquarium["Especie"].map(get_habitat)
```

	Salas	Especie	Total peces	Hábitat
0	1	Boquerón	10	agua salada
1	2	Trucha	8	agua dulce
2	3	Barbo	15	agua dulce
3	4	Sardina	20	agua salada
4	5	Carpa	9	agua dulce

Reemplazar valores

La función *replace()* ofrece otra forma sencilla y flexible de modificar valores.

```
import pandas as pd
import numpy as np
```

```
numeros = pd.Series([1, 3, -999, 0, 2, -1000, -999, 5, 9])
```

```
0      1
1      3
2    -999
3       0
4       2
5   -1000
6    -999
7       5
8       9
dtype: int64
```

```
# -999 y -1000 podrían ser valores centinelas para datos faltantes
# Los reemplazamos para poder tratarlos mejor con pandas
```

```
numeros= numeros.replace([-999,-1000],np.nan)
```

```
0      1.0
1      3.0
2      NaN
3      0.0
4      2.0
5      NaN
6      NaN
7      5.0
8      9.0
dtype: float64
```


Discretización

La discretización es el proceso de convertir datos continuos en categóricos, dividiéndolos en intervalos o grupos. Pandas ofrece dos funciones:

- `cut()`: crea intervalos de tamaño fijo o definidos manualmente
- `qcut()`: que crea intervalos con el mismo número de elementos en cada uno (cuantiles).

```
import pandas as pd
'''
Imaginemos que tenemos datos sobre un grupo de personas
y queremos agruparlos por edad
Lo queremos dividir en tres intervalos:
De 18 a 25
De 26 a 40
De 41 a 60
'''
edad = [20, 49, 29, 28, 31, 26, 47, 32]
grupos = [18, 25, 40, 60]

# Vamos a ponerle nombre a cada grupo
division = pd.cut(edad, grupos,
                  labels=["Joven", "Adulto Joven", "Adulto"])

print(division)

['Joven', 'Adulto', 'Adulto Joven', 'Adulto Joven', 'Adulto Joven', 'Adulto Joven', 'Adulto', 'Adulto Joven']
Categories (3, object): ['Joven' < 'Adulto Joven' < 'Adulto']

#Muestra a qué intervalo pertenece cada elemento de "edad"
print(division.codes)

      [0 2 1 1 1 1 2 1]

#Muestra cuantos elementos hay en cada intervalo
print(division.value_counts())

      Joven      1
      Adulto Joven  5
      Adulto      2
'''
Ahora creamos 4 grupos con la misma cantidad de elementos
precision=2 limita la precisión decimal a 2 dígitos
'''
iguales = pd.qcut(edad, 4, precision=2)
print(iguales)
```

```

[(19.99, 27.5], (35.75, 49.0], (27.5, 30.0], (27.5, 30.0], (30.0, 35.75], (19.99, 27.5], (35.75, 49.0], (30.0, 35.75]]
Categories (4, interval[float64, right]): [(19.99, 27.5] < (27.5, 30.0] < (30.0, 35.75] <
                                           (35.75, 49.0]]

print(iguales.value_counts())

           (19.99, 27.5]    2
           (27.5, 30.0]    2
           (30.0, 35.75]    2
           (35.75, 49.0]    2
Name: count, dtype: int64

```

Detección y filtrado de valores atípicos

Los valores atípicos (outliers) son datos que se alejan significativamente del resto de los valores en un conjunto de datos. Pueden ser errores, datos extremos o información relevante. El método más común para detectarlos es el [rango intercuartílico](#).

```

import pandas as pd
import numpy as np

# Creamos DataFrame con valores atípicos
df = pd.DataFrame({"Ventas": [100, 120, 110, 105, 5000, 130,
                              140, 9000, 150, 160]})

           Ventas
0           100
1           120
2           110
3           105
4          5000
5           130
6           140
7          9000
8           150
9           160

# Calculamos cuartiles y la diferencia
Q1 = df["Ventas"].quantile(0.25)
Q3 = df["Ventas"].quantile(0.75)
IQR = Q3 - Q1

# Calculamos límites para que un valor se considere atípico
limite_inferior = Q1 - 1.5 * IQR

```

```

limite_superior = Q3 + 1.5 * IQR

# Detectamos valores atípicos
outliers = df[(df["Ventas"] < limite_inferior) |
               (df["Ventas"] > limite_superior)]
print(outliers)

```

	Ventas
4	5000
7	9000

```

# Filtramos el dataFrame
df_filtrado = df[(df["Ventas"] >= limite_inferior) &
                  (df["Ventas"] <= limite_superior)]
print(df_filtrado)

```

	Ventas
0	100
1	120
2	110
3	105
5	130
6	140
8	150
9	160

Permutación y muestreo aleatorio

Son técnicas clave en el análisis de datos, especialmente cuando queremos hacer análisis exploratorio, reducir el tamaño de un dataset o validar modelos de Machine Learning. Consiste en reorganizar los datos de forma aleatoria para poder sacar una muestra. Se puede hacer de distintas formas.

```

import pandas as pd
import numpy as np

# Creamos DataFrame con valores del 0 al 34
df = pd.DataFrame(np.arange(35).reshape(5, 7))
print(df)

```

```

      0  1  2  3  4  5  6
0  0  1  2  3  4  5  6
1  7  8  9 10 11 12 13
2 14 15 16 17 18 19 20
3 21 22 23 24 25 26 27
4 28 29 30 31 32 33 34

# Subconjunto aleatorio de 3 filas
print(df.sample(n=3))

      0  1  2  3  4  5  6
3 21 22 23 24 25 26 27
4 28 29 30 31 32 33 34
1  7  8  9 10 11 12 13

# Elegimos el 50% de las columnas
print(df.sample(frac=0.5, axis=1))

      1  2  0  6
0  1  2  0  6
1  8  9  7 13
2 15 16 14 20
3 22 23 21 27
4 29 30 28 34

# Con repeticiones de filas habilitadas
print(df.sample(n=8, replace=True))

      0  1  2  3  4  5  6
2 14 15 16 17 18 19 20
4 28 29 30 31 32 33 34
2 14 15 16 17 18 19 20
2 14 15 16 17 18 19 20
2 14 15 16 17 18 19 20
0  0  1  2  3  4  5  6
4 28 29 30 31 32 33 34
4 28 29 30 31 32 33 34

```

Agrupación de datos

El método **groupby()** de Pandas es una herramienta fundamental para agrupar datos de un DataFrame según una o varias columnas. Es una de las funciones más poderosas y útiles, especialmente cuando toca trabajar con grandes volúmenes de datos que deben ser segmentados en grupos significativos para el análisis.

Vamos a ver un ejemplo de su uso: Supongamos que tenemos un DataFrame con datos de ventas de productos y queremos obtener el total de ventas por categoría:

```
import pandas as pd

df = pd.DataFrame({
    'Producto': ['A', 'B', 'A', 'B', 'C'],
    'Categoria': ['Ropa', 'Ropa', 'Electrónica', 'Electrónica',
'Ropa'],
    'Ventas': [100, 200, 150, 300, 120]
})

# Agrupamos por 'Categoria' y sumamos las ventas
ventas_por_categoria = df.groupby('Categoria')['Ventas'].sum()

print(ventas_por_categoria)
```

Categoria	
Electrónica	450
Ropa	420

Name: Ventas, dtype: int64

Este método es común verlo acompañado de funciones como `sum()`, `mean()`, `count()`, `min()`, `max()`... Se pueden aplicar varias usando `agg()`.

```
df.groupby('Categoria')['Ventas'].agg(['sum', 'mean', 'max'])
```

También permite aplicar funciones propias sobre cada grupo utilizando `apply()` o filtrar grupos que cumplan una condición específica usando `filter()`.

Datos tipo *datetime*

En Pandas, *datetime* es un tipo de dato crucial para el trabajo con series temporales, fechas y horas. Pandas ofrece la función **`pd.to_datetime()`** para convertir cadenas de texto o números en objetos de fecha y hora *datetime*. Una vez convertidos, puedes realizar operaciones como filtrar por rango de fechas, extraer años o meses o calcular diferencias entre fechas.

```
import pandas as pd
```

```
df = pd.DataFrame({
    'Fecha_Venta': ['2021-01-01', '2021-02-01', '2021-03-01'],
    'Ventas': [200, 250, 300]
})
```

```
print(df["Fecha_Venta"]) #Vemos que NO es tipo datetime
```

```
0    2021-01-01
1    2021-02-01
2    2021-03-01
Name: Fecha_Venta, dtype: object
```

```
# Convertimos a tipo datetime
df['Fecha_Venta'] = pd.to_datetime(df['Fecha_Venta'])
print(df["Fecha_Venta"])
```

```
0    2021-01-01
1    2021-02-01
2    2021-03-01
Name: Fecha_Venta, dtype: datetime64[ns]
```

```
# Esto nos permite filtrar por ejemplo por mes
df_filtrado = df[df['Fecha_Venta'].dt.month == 2]
print(df_filtrado)
```

```
      Fecha_Venta  Ventas
1  2021-02-01      250
```

Apartado de ampliación.

Lo que aparece de aquí en adelante no entra en el examen.

Renombrar índices de eje

Igual que los valores de una serie, las etiquetas de eje se pueden cambiar. Renombrar ejes en un DataFrame de Pandas permite mejorar la claridad y comprensión de los datos, facilitando su análisis e interpretación. En muchas ocasiones, los nombres de filas o columnas pueden ser genéricos, poco descriptivos o confusos, por lo que es útil ajustarlos para que reflejen mejor el contenido de la información.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'Date': ['1/1/2021', '1/1/2021', '2/1/2021',
                           '2/1/2021', '1/1/2021', '1/1/2021', '2/1/2021',
                           '2/1/2021'],
                  'City': ['Londres', 'Madrid', 'Tokio',
                           'Liverpool',
                           'Berlín', 'Zafra', 'Badajoz', 'Cáceres'],
                  'Temperature': [25, 29, 28, 31, 26, 27, 22, 32],
                  'Humidity': [46, 50, 52, 59, 42, 45, 46, 43]})
```

	Date	City	Temperature	Humidity
0	1/1/2021	Londres	25	46
1	1/1/2021	Madrid	29	50
2	2/1/2021	Tokio	28	52
3	2/1/2021	Liverpool	31	59
4	1/1/2021	Berlín	26	42
5	1/1/2021	Zafra	27	45
6	2/1/2021	Badajoz	22	46
7	2/1/2021	Cáceres	32	43

```
# Cambiamos filas y columnas con rename
# Pero crea objeto nuevo, no modifica el original
renombrado = df.rename(index={0:"primera"},
                      columns={"Date": "Fecha",
                              "City": "Ciudad",
                              "Temperature": "Temperatura",
                              "Humidity": "Humedad"}
```

```

    })

print(renombrado)


```

	Fecha	Ciudad	Temperatura	Humedad
primera	1/1/2021	Londres	25	46
1	1/1/2021	Madrid	29	50
2	2/1/2021	Tokio	28	52
3	2/1/2021	Liverpool	31	59
4	1/1/2021	Berlín	26	42
5	1/1/2021	Zafra	27	45
6	2/1/2021	Badajoz	22	46
7	2/1/2021	Cáceres	32	43

```

# Podemos cambiar las filas también podemos con map
# En este caso sí se modifica el original
nuevo_indice = {
    "primera": "Enero_1",
    1: "Enero_2",
    2: "Febrero_1",
    3: "Febrero_2",
    4: "Enero_3",
    5: "Enero_4",
    6: "Febrero_3",
    7: "Febrero_4"
}

def indice_mes(x):
    return nuevo_indice[x]

renombrado.index = renombrado.index.map(nuevo_indice)


```

	Fecha	Ciudad	Temperatura	Humedad
Enero_1	1/1/2021	Londres	25	46
Enero_2	1/1/2021	Madrid	29	50
Febrero_1	2/1/2021	Tokio	28	52
Febrero_2	2/1/2021	Liverpool	31	59
Enero_3	1/1/2021	Berlín	26	42
Enero_4	1/1/2021	Zafra	27	45
Febrero_3	2/1/2021	Badajoz	22	46
Febrero_4	2/1/2021	Cáceres	32	43

Calcular variables dummy o indicadoras

Una variable dummy es aquella que toma el valor 1 o 0 para indicar la presencia o ausencia de una cierta característica o condición. Se utiliza comúnmente en análisis estadísticos y modelos de regresión para incluir información categórica en un modelo matemático. Al cambiar los datos categóricos, que son difíciles de entender para los algoritmos, a un formato numérico nos permite agrupar los datos categóricos sin perder ninguna información.

```
import pandas as pd
import numpy as np

#Creamos dataframe con datos aleatorios
df = pd.DataFrame(np.random.rand(10, 2).round(3) * 10,
                  columns=['Precio', 'Cantidad'])
#Añadimos columna con cadenas de texto que tengamos que convertir a dummy
df['Ciudad'] = np.random.choice(['Chicago', 'Boston', 'Nueva York'],
                                size=df.shape[0])
print(df)
```

	Precio	Cantidad	Ciudad
0	5.95	1.98	Boston
1	3.31	4.73	Boston
2	8.14	7.14	Nueva York
3	8.43	6.64	Boston
4	6.90	0.17	Nueva York
5	8.29	1.38	Chicago
6	5.30	7.43	Chicago
7	0.82	2.67	Nueva York
8	6.11	0.90	Boston
9	6.42	5.83	Nueva York

```
#Creamos la variable dummy
dummies = pd.get_dummies(df['Ciudad']).astype(int)

#Quitamos la columna original y metemos la dummy
df = df.drop('Ciudad', axis=1)
df = df.join(dummies)
print(df)
```

	Precio	Cantidad	Boston	Chicago	Nueva York
0	5.95	1.98	1	0	0
1	3.31	4.73	1	0	0
2	8.14	7.14	0	0	1
3	8.43	6.64	1	0	0
4	6.90	0.17	0	0	1
5	8.29	1.38	0	1	0
6	5.30	7.43	0	1	0
7	0.82	2.67	0	0	1
8	6.11	0.90	1	0	0
9	6.42	5.83	0	0	1

Manipulación de cadenas de texto

Como programadores, es común que necesitemos trabajar con cadenas de texto o strings en nuestro código. Python facilita la manipulación de strings con una gran cantidad de herramientas y funciones.

En primer lugar vamos a ver los métodos de cadenas de texto internos de Python.

Método	Descripción
count	Cuenta las veces que aparece en una cadena de texto la subcadena que especificamos como argumento
endswith	Devuelve True si la cadena de texto termina con la cadena que especificamos
startswith	Devuelve True si la cadena de texto empieza con la cadena que especificamos
join	Une los elementos de una secuencia con el string que llama al método como separador.
index/find	Devuelve la posición de la primera aparición de la subcadena de texto en la cadena donde se busca
rfind	Devuelve la posición de la última aparición de la subcadena de texto en la cadena donde se busca
replace	Sustituye las apariciones de la cadena de texto por otra cadena de texto
strip,rstrip,lstrip	Quita espacios en blanco incluyendo nuevas líneas en ambos lados, en el lado derecho o en el lado izquierdo

split	Divide la cadena de texto en listas de subcadenas utilizando el delimitador que se especifica
lower	Pasa todos los caracteres alfabéticos a minúsculas
upper	Pasa todos los caracteres alfabéticos a mayúsculas
ljust, rjust	Justifica a la izquierda o a la derecha, respectivamente, y rellena con espacios o un carácter especificado hasta que la cadena alcanza una longitud dada.

Vamos a ver algunos ejemplos de uso de estos métodos. En los comentarios aparece la salida que se produciría para evitar tener que poner tantas capturas de pantalla al tratarse de cadenas de texto.

```

texto = "Hola Mundo"

print(texto.lower())    # hola mundo
print(texto.upper())    # HOLA MUNDO

print(texto.startswith("Hola"))    # True
print(texto.endswith("Mundo"))    # True

print(texto.count("o"))    # 2 (cuántas veces aparece 'o')
print(texto.find("Mundo")) # 5 (posición de "Mundo")
print(texto.rfind("o"))    # 9 (última aparición de 'o')

print(texto.ljust(15, "-")) # "Hola Mundo----"
print(texto.rjust(15, "-")) # "----Hola Mundo"

lista = ["Python", "es", "genial"]
print(" ".join(lista))    # "Python es genial"
print(texto.split())      # ['Hola', 'Mundo']

print(texto.replace("Mundo", "Python")) # Hola Python

```

A la hora de buscar en un texto patrones de cadenas de texto o coincidencias, se utilizan las expresiones regulares (a menudo abreviadas como regex). Python usa el módulo `re` para trabajar con ellas. En Python, las expresiones regulares están soportadas por el módulo `re` que tenemos que importar. En la siguiente tabla aparecen métodos de expresiones regulares:

Método	Descripción
findall	Devuelve todos los patrones de coincidencia de un texto
match	Devuelve un objeto coincidente si el texto coincide con el patrón del principio. En caso contrario, devuelve None.
search	Recorre la cadena/secuencia dada, buscando la primera posición en la que se produzca una coincidencia con el patrón
split	Divide las cadenas donde coincida el patrón y devuelve una lista.
sub/subn	Reemplaza todas o las n primeras apariciones del patrón en la cadena de texto con la expresión de reemplazo

Caracteres especiales en expresiones regulares

Símbolo	Significado
.	Cualquier caracter
\d	Dígitos (0-9)
\w	Letras, números o guión bajo
\s	Espacio en blanco
+	Uno o más
*	Cero o más
?	Cero o uno
{n,m}	Repetición entre n y m veces
^	Inicio de línea
\$	Fin de línea

Para evitar que algunos símbolos (por ejemplo los que empiezan por \) se malinterpreten como comandos especiales, usaremos r" "

```
import re

texto = "Mi número es 654-789-123"
patron = r"\d{3}-\d{3}-\d{3}"
# Busca un número en formato XXX-XXX-XXX
```

```
match = re.search(patron, texto)
if match:
    print("Número encontrado:", match.group()) # 654-789-123

texto = "Los valores son 20, 30 y 50"
patron = r"\d+"
# Busca uno o más dígitos
numeros = re.findall(patron, texto)
print(numeros) # ['20', '30', '50']

texto = "Mi correo es roberto23@gmail.com"
patron = r"\w+@\w+\. \w+"
# Sustituimos correos electrónicos
texto_oculto = re.sub(patron, "[EMAIL OCULTO]", texto)
print(texto_oculto) # "Mi correo es [EMAIL OCULTO]"

texto = "Nombre: Juan, Edad: 30, País: España"
patron = r",\s*"
# Divide por comas seguidas de espacios
partes = re.split(patron, texto)
print(partes) # ['Nombre: Juan', 'Edad: 30', 'País: España']
```

En la documentación de Python se pueden consultar todos los métodos de cadena de texto que hay. Pincha en [este enlace](#) para verlos.

Table of Contents

Built-in Types

- Truth Value Testing
- Boolean Operations — **and**, **or**, **not**
- Comparisons
- Numeric Types — **int**, **float**, **complex**
 - Bitwise Operations on Integer Types
 - Additional Methods on Integer Types
 - Additional Methods on Float
 - Hashing of numeric types
- Boolean Type - **bool**
- Iterator Types
 - Generator Types
- Sequence Types — **list**, **tuple**, **range**
 - Common Sequence Operations
 - Immutable Sequence Types
 - Mutable Sequence Types
 - Lists
 - Tuples
 - Ranges
- Text Sequence Type — **str**
 - **String Methods**
 - **printf**-style String Formatting
- Binary Sequence Types — **bytes**, **bytearray**, **memoryview**

Datos categóricos

Los datos categóricos son valores que pertenecen a un conjunto limitado de categorías.
Por ejemplo:

- Género: "Masculino", "Femenino", "No Binario"
- Colores: "Rojo", "Azul", "Verde", "Amarillo", "Rosa"
- Clasificación de productos: "Básico", "Premium", "VIP"

Para su representación en pandas, se usa el tipo de dato Categorical que optimiza memoria y mejora el rendimiento.

```
import pandas as pd

df = pd.DataFrame({"Mes": ["Enero", "Marzo", "Mayo", "Marzo", "Abril"]})

# Convertimos la columna en tipo categórico
df["Mes"] = df["Mes"].astype("category")

print(df.dtypes)

           Mes    category
dtype: object
```

Al poner la columna como categórica se conseguimos un menor uso de memoria (útil en grandes datasets) y más rapidez en operaciones y filtros. Además, permite definir un orden en las categorías (Ejemplo: "Básico" < "Premium" < "VIP").

```
orden = ["Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio"]
df["Mes"] = pd.Categorical(df["Mes"], categories=orden, ordered=True)

#Ahora podemos hacer comparaciones
df["Mes"][2] > df["Mes"][4]
if True:
    print(df["Mes"][2], "es mayor que", df["Mes"][4])

           Mayo es mayor que Abril
```